ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ИМЕНИ ПАТРИСА ЛУМУМБЫ

Отчет
о выполнении лабораторной работы № 1
"Персептрон Розенблатта"
по дисциплине "Нейронные сети"

Вариант задания № 12

Выполнил студент
группы ЗФИбд-01-24
Князев А. М.
_____

Проверил и принял
Леонов С. С.
_____

с оценкой _____

Москва, 2026

Цель работы: исследование свойств персептрона Розенблатта и его применение для решения задачи распознавания образов.
Основные этапы работы:

1. Для первой обучающей выборки построить и обучить сеть, которая будет правильно относить точки к двум классам. Отобразить дискриминантную линию и проверить качество обучения.

2. Изменить обучающее множество так, чтобы классы стали линейно неразделимыми. Проверить возможности обучения по правилу Розенблатта.

3. Для второй обучающей выборки построить и обучить сеть, которая будет правильно относить точки к четырем классам. Отобразить дискриминантную линию и проверить качество обучения.

Оборудование: персональный компьютер. Программное обеспечение: Python 3.x, Jupyter Notebook, библиотеки NumPy и Matplotlib.

Сценарий выполнения работы

1) Бинарная классификация (обучающая выборка из 6 точек)

Задана обучающая выборка из 6 точек плоскости в диапазоне [-5; 5], разделённых на 2 класса. Координаты точек и их метки приведены в таблице 1.

Таблица 1 — Обучающая выборка (бинарная классификация)

№ точки	x1	x2	Класс
1	2.7	4.3	0
2	-3.8	0.6	0
3	-0.4	-4.9	1
4	-1.7	-3.4	1
5	2.9	-1.9	1
6	0.2	-3.4	1

Структура сети: однонейронный персептрон с 2 входами (x1, x2), пороговой функцией активации hardlim и ненулевым смещением (bias). Нейрон имеет 2 весовых коэффициента и 1 смещение.

Ручное обучение (2 эпохи):

Веса и смещение инициализированы случайными значениями из равномерного распределения [-1, 1] (аналог функции rands в MATLAB). Начальные значения (seed=12): W = [-0.692, 0.480], b = [-0.473].

Обучение выполнялось по правилу Розенблатта: для каждого образца вычисляется выход сети, ошибка $e = t - y$, и при ненулевой ошибке обновляются веса $W = W + lr \cdot e \cdot p^T$ и смещение $b = b + lr \cdot e$. Скорость обучения lr = 0.5.

Результат: сеть сошлась за 1 эпоху, MAE = 0.0000. После 2-й эпохи веса не изменились. Финальные веса: W = [1.008, -2.270], b = [-0.473].

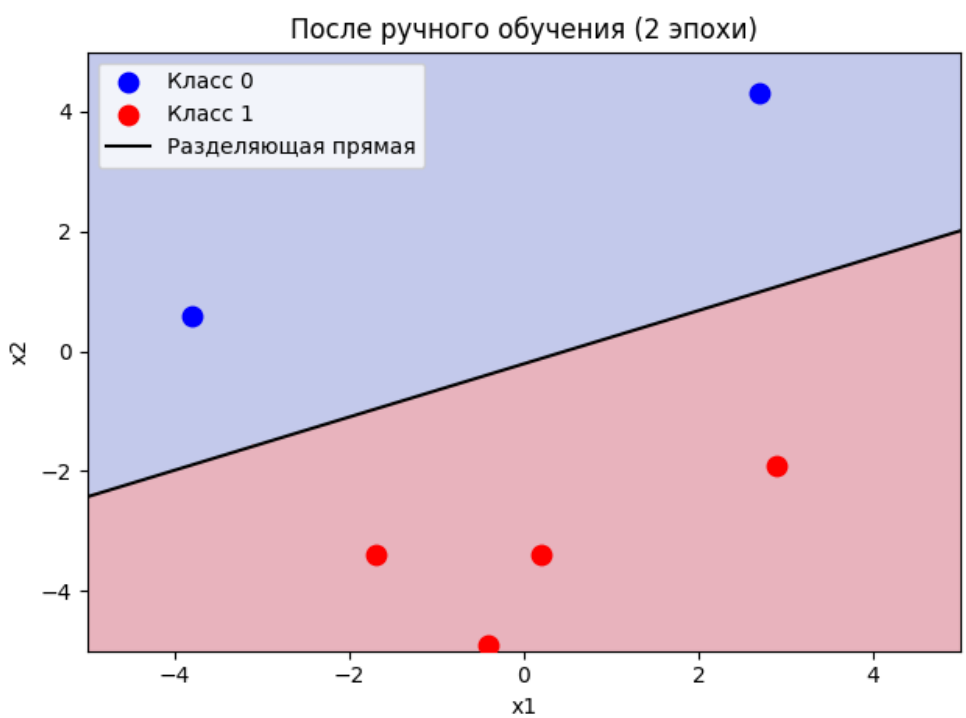

Рис. 1 — Результат ручного обучения (2 эпохи)

Автоматическое обучение (50 эпох):

Веса повторно инициализированы случайными значениями (seed=15). Обучение проведено функцией rosenblatt_train с числом эпох 50.

Финальные весовые коэффициенты: W = [0.698, -0.642], b = -0.891. MAE на обучающей выборке = 0.0000.

Проверка качества: случайным образом сгенерированы 3 тестовые точки (seed=33, диапазон [-5; 5]). Все 3 точки классифицированы корректно.

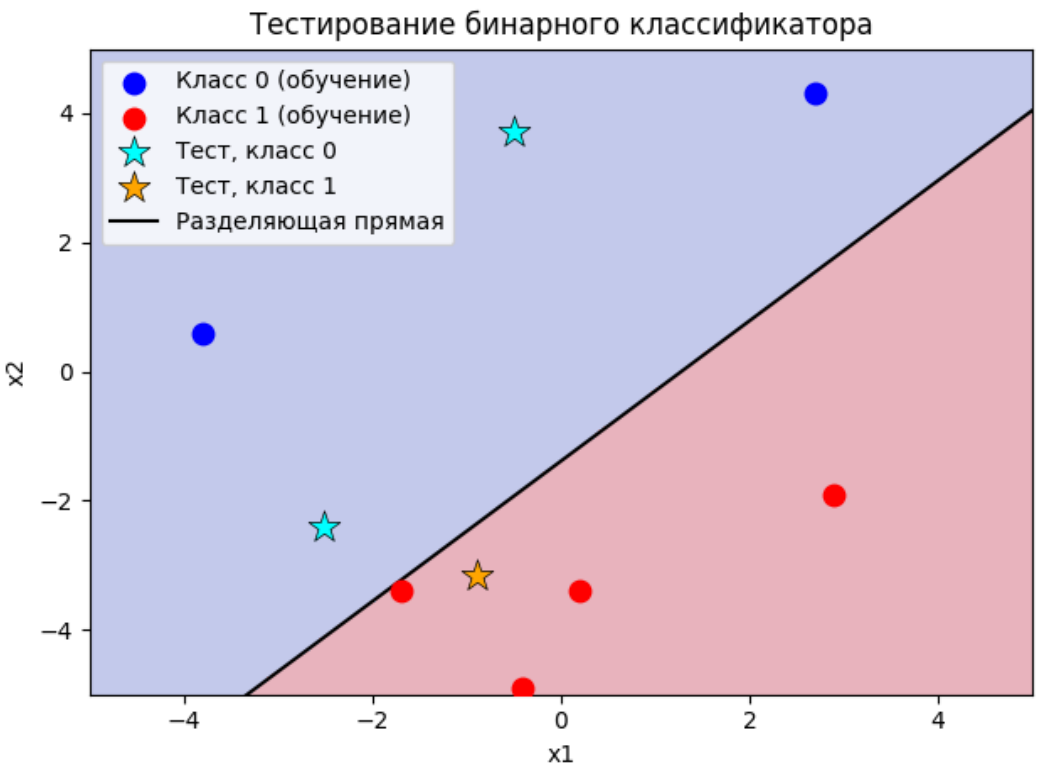

Рис. 2 — Тестирование бинарного классификатора

2) Линейно неразделимый случай

Обучающая выборка модифицирована добавлением точки (-1.0, -3.0) класса 0 в область, принадлежащую классу 1 в исходной конфигурации. Модифицированная выборка содержит 7 точек и является линейно неразделимой.

После 50 эпох обучения (seed=20): MAE = 0.1429, что подтверждает невозможность линейного разделения. Персептрон Розенблатта не может найти разделяющую гиперплоскость для линейно неразделимых данных, что соответствует теоретическим ограничениям однослойного персептрона.

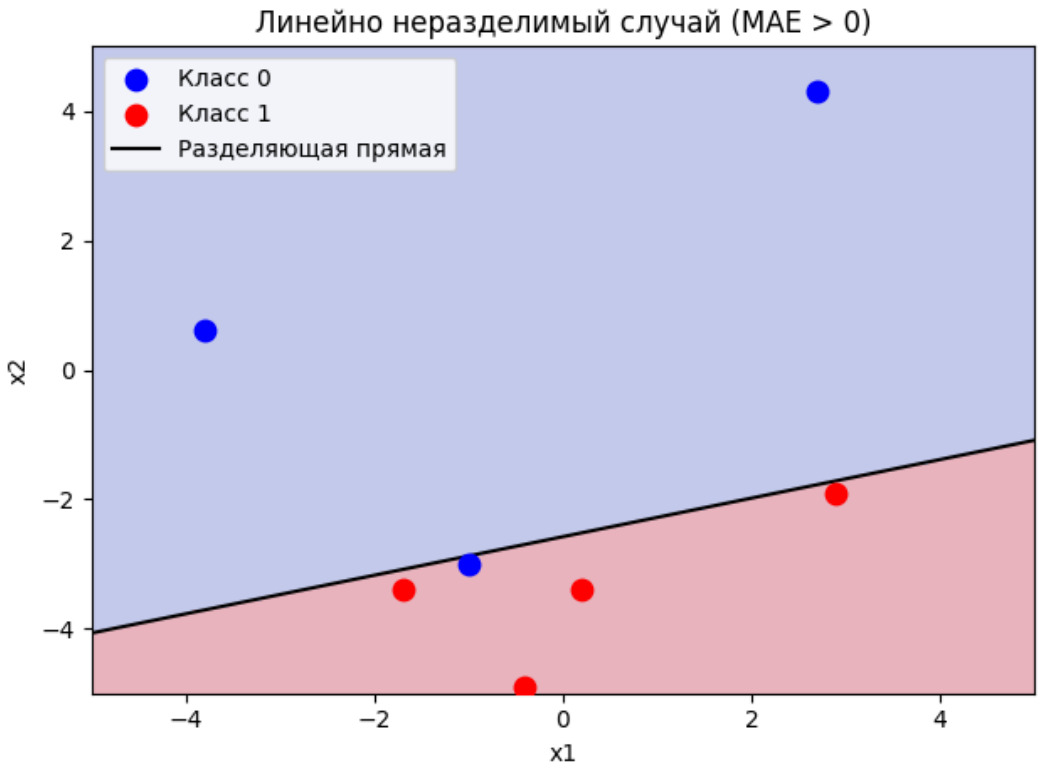

Рис. 3 — Линейно неразделимый случай

3) Классификация на 4 класса (2 нейрона)

Задана вторая обучающая выборка из 8 точек, разделённых на 4 класса. Метки классов представлены 2-битным вектором [t1, t2]. Координаты и метки приведены в таблице 2.

Таблица 2 — Обучающая выборка (4 класса)

№	x1	x2	t1	t2
1	-1.5	-0.6	0	0
2	4.6	-4.6	0	1
3	4.7	-3.2	0	1
4	1.6	0.8	1	0
5	1.7	-1.4	0	0
6	1.2	3.1	1	0
7	-4.9	-4.2	0	1
8	4.7	1.5	1	1

Структура сети: персептрон с 2 нейронами (каждый с 2 входами, пороговой функцией активации hardlim и смещением). Каждый нейрон формирует один бит 2-битной метки класса.

Обучение проводилось многократно с различными начальными весами (перебор seed от 0 до 99). Лучший результат: MAE = 0.0625. Данные второй выборки для нейрона t2 являются линейно неразделимыми (точки класса 1 расположены на обоих краях по x1, а класса 0 — в середине), поэтому достичь MAE = 0 невозможно.

Финальные весовые коэффициенты: W = [[-0.760, 2.865], [0.708, -1.228]], b = [-0.178, -0.765].

Проверка качества: случайным образом сгенерированы 5 тестовых точек (seed=55). Результаты классификации визуализированы на графике с двумя дискриминантными линиями (по одной на каждый нейрон) и цветовой сеткой.

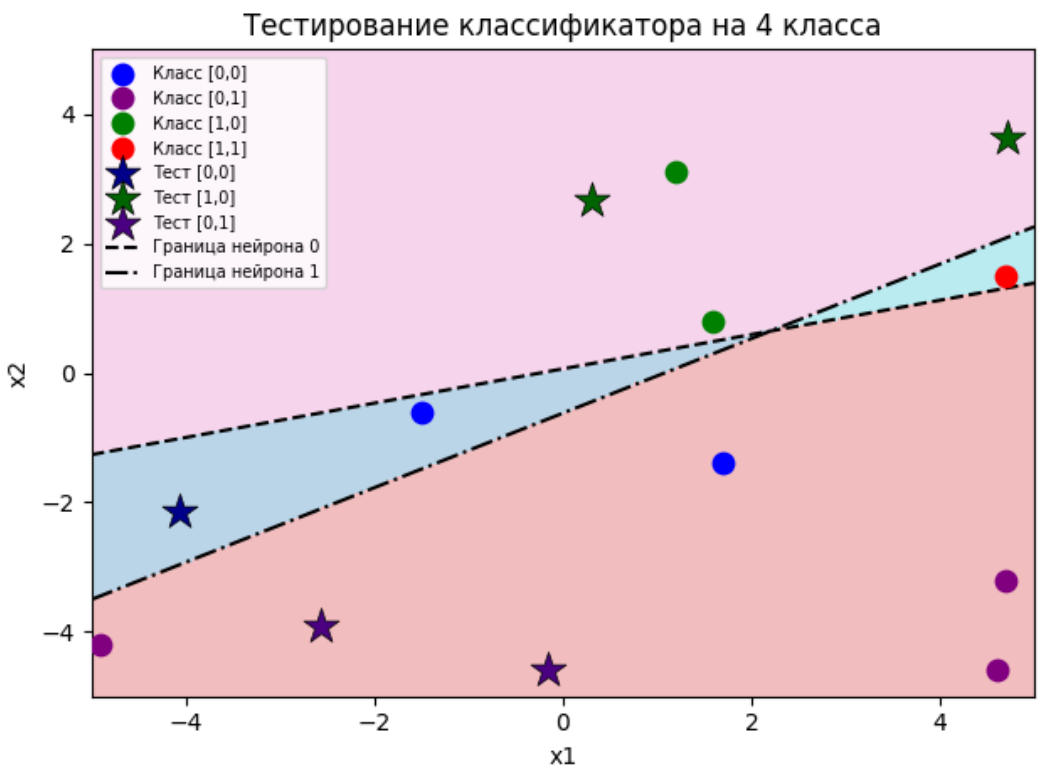

Рис. 4 — Тестирование классификатора на 4 класса

Код программы

```
import numpy as np
import matplotlib.pyplot as plt

# Вспомогательные функции

def hardlim(n):
    return (n >= 0).astype(float)

def sim(P, W, b):
    return hardlim(W @ P + b.reshape(-1, 1))

def mae(x):
    return np.mean(np.abs(x))

def rosenblatt_train(P, T, W, b, epochs=50, lr=0.5):
    Q = P.shape[1]
    for ep in range(epochs):
        for j in range(Q):
            p = P[:, j : j + 1]
            t = T[:, j : j + 1]
            y = hardlim(W @ p + b.reshape(-1, 1))
            e = t - y
            if mae(e) != 0:
                W = W + lr * (e @ p.T)
```

```
                b = b + lr * e.flatten()
        if mae(T - sim(P, W, b)) == 0:
            break
    return W, b

P1 = np.array([[2.7, -3.8, -0.4, -1.7, 2.9, 0.2],
               [4.3,  0.6, -4.9, -3.4, -1.9, -3.4]])
T1 = np.array([[0.0, 0.0, 1.0, 1.0, 1.0, 1.0]])

print("P1:", P1)
print("T1:", T1)

print("=== Структура сети ===")
print(f"Количество входов: 2")
print(f"Количество нейронов: 1")
print(f"Функция активации: hardlim")
print(f"Смещение (bias): да")

np.random.seed(12)
W1 = np.random.uniform(-1, 1, (1, 2))
b1 = np.random.uniform(-1, 1, 1)

print("W начальные:", W1)
print("b начальное:", b1)

lr = 0.5
Q1 = P1.shape[1]

for ep in range(1, 3):
    for j in range(Q1):
        p = P1[:, j : j + 1]
        t = T1[:, j : j + 1]
        y = hardlim(W1 @ p + b1.reshape(-1, 1))
        e = t - y
        if mae(e) != 0:
            W1 = W1 + lr * (e @ p.T)
            b1 = b1 + lr * e.flatten()
    Y1 = sim(P1, W1, b1)
    print(f"Эпоха {ep}: W={W1}, b={b1}, MAE={mae(T1 - Y1):.4f}")

x1_range = np.linspace(-5, 5, 200)
x2_line = -(W1[0, 0] * x1_range + b1[0]) / W1[0, 1]

xx1, xx2 = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5,
200))
grid_points = np.vstack([xx1.ravel(), xx2.ravel()])
grid_classes = sim(grid_points, W1, b1).reshape(xx1.shape)

fig, ax = plt.subplots()
ax.pcolormesh(xx1, xx2, grid_classes, alpha=0.3, cmap="coolwarm")
colors = {0.0: "blue", 1.0: "red"}
labels = {0.0: "Класс 0", 1.0: "Класс 1"}
plotted = set()
for i in range(P1.shape[1]):
    cls = T1[0, i]
```

```
    label = labels[cls] if cls not in plotted else None
    ax.scatter(P1[0, i], P1[1, i], color=colors[cls], label=label,
s=80, zorder=3)
    plotted.add(cls)
ax.plot(x1_range, x2_line, "k-", label="Разделяющая прямая")
ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("После ручного обучения (2 эпохи)")
ax.legend()
plt.tight_layout()
plt.show()

np.random.seed(15)
W1 = np.random.uniform(-1, 1, (1, 2))
b1 = np.random.uniform(-1, 1, 1)

W1, b1 = rosenblatt_train(P1, T1, W1, b1, epochs=50, lr=0.5)

Y1 = sim(P1, W1, b1)
print("W финальные:", W1)
print("b финальное:", b1)
print(f"MAE на обучающей выборке: {mae(T1 - Y1):.4f}")

np.random.seed(33)
P_test = np.random.uniform(-5, 5, (2, 3))
T_test = sim(P_test, W1, b1)

for i in range(3):
    print(f"Точка {i+1}: x1={P_test[0,i]:.3f}, x2={P_test[1,i]:.3f} ->
класс {int(T_test[0,i])}")

xx1, xx2 = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5,
200))
grid_points = np.vstack([xx1.ravel(), xx2.ravel()])
grid_classes = sim(grid_points, W1, b1).reshape(xx1.shape)

x1_range = np.linspace(-5, 5, 200)
x2_line = -(W1[0, 0] * x1_range + b1[0]) / W1[0, 1]

fig, ax = plt.subplots()
ax.pcolormesh(xx1, xx2, grid_classes, alpha=0.3, cmap="coolwarm")

colors_train = {0.0: "blue", 1.0: "red"}
labels_train = {0.0: "Класс 0 (обучение)", 1.0: "Класс 1 (обучение)"}
plotted = set()
for i in range(P1.shape[1]):
    cls = T1[0, i]
    label = labels_train[cls] if cls not in plotted else None
    ax.scatter(P1[0, i], P1[1, i], color=colors_train[cls],
label=label, s=80, zorder=3)
    plotted.add(cls)

colors_test = {0.0: "cyan", 1.0: "orange"}
```

```
labels_test = {0.0: "Тест, класс 0", 1.0: "Тест, класс 1"}
plotted_test = set()
for i in range(P_test.shape[1]):
    cls = T_test[0, i]
    label = labels_test[cls] if cls not in plotted_test else None
    ax.scatter(P_test[0, i], P_test[1, i], color=colors_test[cls],
marker="*",
               s=200, zorder=4, edgecolors="black", linewidths=0.5,
label=label)
    plotted_test.add(cls)

ax.plot(x1_range, x2_line, "k-", label="Разделяющая прямая")
ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Тестирование бинарного классификатора")
ax.legend()
plt.tight_layout()
plt.show()

# Добавляем точку (-1.0, -3.0) класса 0 в область класса 1 — нарушаем
линейную разделимость
P_mod = np.hstack([P1, np.array([[-1.0], [-3.0]])])
T_mod = np.hstack([T1, np.array([[0.0]])])

print("P_mod:", P_mod)
print("T_mod:", T_mod)

np.random.seed(20)
W_mod = np.random.uniform(-1, 1, (1, 2))
b_mod = np.random.uniform(-1, 1, 1)

W_mod, b_mod = rosenblatt_train(P_mod, T_mod, W_mod, b_mod, epochs=50,
lr=0.5)

Y_mod = sim(P_mod, W_mod, b_mod)
print("W финальные:", W_mod)
print("b финальное:", b_mod)
print(f"MAE на модифицированной выборке: {mae(T_mod - Y_mod):.4f}")

x1_range = np.linspace(-5, 5, 200)
x2_line_mod = -(W_mod[0, 0] * x1_range + b_mod[0]) / W_mod[0, 1]

xx1, xx2 = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5,
200))
grid_points = np.vstack([xx1.ravel(), xx2.ravel()])
grid_classes_mod = sim(grid_points, W_mod, b_mod).reshape(xx1.shape)

fig, ax = plt.subplots()
ax.pcolormesh(xx1, xx2, grid_classes_mod, alpha=0.3, cmap="coolwarm")

colors_cls = {0.0: "blue", 1.0: "red"}
labels_cls = {0.0: "Класс 0", 1.0: "Класс 1"}
plotted = set()
```

```
for i in range(P_mod.shape[1]):
    cls = T_mod[0, i]
    label = labels_cls[cls] if cls not in plotted else None
    ax.scatter(P_mod[0, i], P_mod[1, i], color=colors_cls[cls],
label=label, s=80, zorder=3)
    plotted.add(cls)

ax.plot(x1_range, x2_line_mod, "k-", label="Разделяющая прямая")
ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Линейно неразделимый случай (MAE > 0)")
ax.legend()
plt.tight_layout()
plt.show()

P2 = np.array([[-1.5, 4.6, 4.7, 1.6, 1.7, 1.2, -4.9, 4.7],
               [-0.6, -4.6, -3.2, 0.8, -1.4, 3.1, -4.2, 1.5]])
T2 = np.array([[0.0, 0.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0],
               [0.0, 1.0, 1.0, 0.0, 0.0, 0.0, 1.0, 1.0]])

print("P2:", P2)
print("T2:", T2)

print("\n=== Структура сети ===")
print(f"Количество входов: 2")
print(f"Количество нейронов: 2")
print(f"Функция активации: hardlim")
print(f"Смещение (bias): да")

best_mae = 1.0
best_W, best_b = None, None

max_attempts = 100
for attempt in range(max_attempts):
    np.random.seed(attempt)
    W2 = np.random.uniform(-1, 1, (2, 2))
    b2 = np.random.uniform(-1, 1, 2)
    W2, b2 = rosenblatt_train(P2, T2, W2, b2, epochs=50, lr=0.5)
    Y2 = sim(P2, W2, b2)
    current_mae = mae(T2 - Y2)
    if current_mae < best_mae:
        best_mae = current_mae
        best_W, best_b = W2.copy(), b2.copy()
    if current_mae == 0:
        print(f"Сошлось за попытку {attempt + 1} (seed={attempt})")
        break

W2, b2, final_mae = best_W, best_b, best_mae

if final_mae != 0:
    print(f"Не удалось достичь MAE=0 за {max_attempts} попыток.")

print("W2 финальные:", W2)
```

```
print("b2 финальные:", b2)
print(f"MAE на обучающей выборке: {final_mae:.4f}")

np.random.seed(55)
P_test2 = np.random.uniform(-5, 5, (2, 5))
T_test2 = sim(P_test2, W2, b2)

for i in range(5):
    print(f"Точка {i+1}: x1={P_test2[0,i]:.3f}, x2={P_test2[1,i]:.3f} -
> метка [{int(T_test2[0,i])}, {int(T_test2[1,i])}]")

xx1, xx2 = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5,
200))
grid_points = np.vstack([xx1.ravel(), xx2.ravel()])
grid_out = sim(grid_points, W2, b2)
grid_label = (grid_out[0] * 2 + grid_out[1]).reshape(xx1.shape)

fig, ax = plt.subplots()
ax.pcolormesh(xx1, xx2, grid_label, alpha=0.3, cmap="tab10", vmin=0,
vmax=3)

class_colors = {(1, 1): "red", (0, 0): "blue", (1, 0): "green", (0, 1):
"purple"}
class_labels = {(1, 1): "Класс [1,1]", (0, 0): "Класс [0,0]",
                (1, 0): "Класс [1,0]", (0, 1): "Класс [0,1]"}
plotted_train = set()
for i in range(P2.shape[1]):
    cls = (int(T2[0, i]), int(T2[1, i]))
    label = class_labels[cls] if cls not in plotted_train else None
    ax.scatter(P2[0, i], P2[1, i], color=class_colors[cls], s=80,
zorder=3, label=label)
    plotted_train.add(cls)

test_colors = {(1, 1): "darkred", (0, 0): "darkblue", (1, 0):
"darkgreen", (0, 1): "indigo"}
test_labels = {(1, 1): "Тест [1,1]", (0, 0): "Тест [0,0]",
               (1, 0): "Тест [1,0]", (0, 1): "Тест [0,1]"}
plotted_test = set()
for i in range(P_test2.shape[1]):
    cls = (int(T_test2[0, i]), int(T_test2[1, i]))
    label = test_labels[cls] if cls not in plotted_test else None
    ax.scatter(P_test2[0, i], P_test2[1, i], color=test_colors[cls],
marker="*",
               s=250, zorder=4, edgecolors="black", linewidths=0.5,
label=label)
    plotted_test.add(cls)

x1_range = np.linspace(-5, 5, 200)
for i, (ls, lbl) in enumerate([("k--", "Граница нейрона 0"), ("k-.",
"Граница нейрона 1")]):
    if abs(W2[i, 1]) > 1e-9:
        x2_line = -(W2[i, 0] * x1_range + b2[i]) / W2[i, 1]
        ax.plot(x1_range, x2_line, ls, label=lbl)

ax.set_xlim(-5, 5)
```

```
ax.set_ylim(-5, 5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Тестирование классификатора на 4 класса")
ax.legend(loc="upper left", fontsize=7)
plt.tight_layout()
plt.show()
```

Вывод

Персептрон Розенблатта успешно решает задачу классификации для линейно разделимых данных в бинарном случае. При нарушении линейной разделимости алгоритм не сходится (MAE > 0 после 50 эпох), что подтверждает теоретические ограничения однослойного персептрона.

Для второй обучающей выборки (4 класса) задача для нейрона t2 оказалась линейно неразделимой — точки класса 1 расположены на противоположных краях пространства признаков, что делает невозможным их разделение одной прямой. Лучший достигнутый MAE = 0.0625.

Все этапы лабораторной работы выполнены. Графики и численные результаты соответствуют теоретическим ожиданиям.